

Level 3 Documentation: Autonomous Navigation & Nav2 Integration

1. Overview & Objective

The objective of Level 3 is to transition from manual flight and passive mapping (SLAM) to **fully autonomous navigation**. Using the 2D/3D map generated in Level 2, the drone integrates **ROS 2 Navigation2 (Nav2)** stack, global/local costmaps, and path planners to autonomously navigate through the maze without human intervention.

2. System Architecture & Tech Stack

- **Operating System:** Windows Subsystem for Linux (WSL 2 - Ubuntu 22.04 LTS)
- **Middleware:** ROS 2 Humble Hawksbill
- **Simulation:** Gazebo Harmonic ([gz sim](#))
- **Autopilot & Control:** ArduPilot SITL + MAVProxy + ROS 2 Twist/Velocity Bridge
- **Navigation Stack:** ROS 2 Nav2 ([nav2_bringup](#), [dwb_controller](#), [smac_planner](#))

3. Step-by-Step Implementation Guide

Step 1: Install Nav2 Packages

Ensure all required ROS 2 Navigation2 packages are installed in your workspace environment:

Shell

```
sudo apt update
sudo apt install -y ros-humble-navigation2
ros-humble-nav2-bringup ros-humble-slam-toolbox
```

Step 2: Launch Gazebo Simulation & LiDAR Bridge (Terminal 1)

Start the maze environment and ensure LiDAR point clouds are publishing correctly:

Shell

```
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
export GZ_VERSION=harmonic
export GZ_RENDERING_ENGINE=ogre
ros2 launch ardupilot_gz_bringup soham_maze.launch.py
```

Step 3: Start ArduPilot SITL & MAVProxy (Terminal 2 & 3)

Boot the vehicle and connect telemetry for velocity and position control:

Shell

```
# Terminal 2 (SITL Binary)
cd ~/ardupilot
./build/sitl/bin/arducopter --model gazebo-iris --speedup 1
--sim-address=127.0.0.1 -I0
```

Shell

```
# Terminal 3 (MAVProxy)
mavproxy.py --master=tcp:127.0.0.1:5760 --out 172.28.48.1:14550
--map --console
```

In MAVProxy:

Shell

```
mode guided
arm throttle
takeoff 2
```

Step 4: Launch Nav2 and Costmaps (Terminal 4)

Load the navigation stack configured with the maze's costmaps and planners:

Shell

```
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
export GZ_VERSION=harmonic
ros2 launch nav2_bringup navigation_launch.py use_sim_time:=true
```

Step 5: Send Autonomous Goals via RViz or Code

1. Open RViz2 to visualize the global costmap, local costmap, and global planner path.
2. Use the **"Nav2 Goal"** tool in RViz to point a target location inside the maze, or write a python node using `nav2_simple_commander` to dispatch automated waypoints.

Setup Notes & Troubleshooting for Level 3

- The avoidance script file did not exist on first run (the heredoc paste had not been applied); recreated and checked with `python3 -m py_compile`.
- The script needs `output add 127.0.0.1:14560` typed at the MAVProxy prompt, or it waits forever for a heartbeat.
- `arm throttle` failed with `Arm: Leaning`. The drone was tilted in Gazebo after the obstacle was spawned. I checked `status ATTITUDE`, reset the drone upright with `set_pose`, and re-ordered the run (take off first, place the obstacle afterwards). Exact cause not confirmed.



